```c
/*
 1. Link: https://github.com/Neo-Vorsatz-UCT/EEE3096S-2025
 2. Group Number: 64
 3. Members: VRSNEO001
*/

/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "stm32f4xx.h"
#include "lcd_stm32f4.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
// TODO: Add values for below variables
#define NS 1024       // Number of samples in LUT
#define TIM2CLK 16000000  // STM Clock frequency: Hint You might want to check the ioc file
#define F_SIGNAL 0.5        // Frequency of output analog signal

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */
```

```c
/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
DMA_HandleTypeDef hdma_tim2_ch1;

/* USER CODE BEGIN PV */
// TODO: Add code for global variables, including LUTs
const uint32_t Sin_LUT[NS] = {2048,2060,2073,2085,2098,2110,2123,2135,2148,2161,2173,2186,2198,2211,2223,2236,
const uint32_t Saw_LUT[NS] = {0,4,8,12,16,20,24,28,32,36,40,44,48,52,56,60,64,68,72,76,80,84,88,92,96,100,104,
const uint32_t Triangle_LUT[NS] = {0,8,16,24,32,40,48,56,64,72,80,88,96,104,112,120,128,136,144,152,160,168,17
const uint32_t Piano_LUT[NS] = {2304,2304,2304,2304,2304,2304,2305,2304,2306,2307,2305,2306,2307,1816,1775,120
const uint32_t Guitar_LUT[NS] = {1868,1868,1868,1868,1868,1868,1868,1868,1868,1868,1868,1868,1868,1870,1863,18
const uint32_t Drum_LUT[NS] = {2047,2047,2047,2047,2048,2046,2045,2042,2050,2050,2048,2049,2042,1855,2239,2632
uint32_t prev_tick = 0;
uint32_t waveform = 0;



// TODO: Equation to calculate TIM2_Ticks
uint32_t TIM2_Ticks = TIM2CLK/(F_SIGNAL*NS); // How often to write new LUT value
uint32_t DestAddress = (uint32_t) &(TIM3->CCR3); // Write LUT TO TIM3->CCR3 to modify PWM duty cycle


/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM3_Init(void);
/* USER CODE BEGIN PFP */
void EXTI0_IRQHandler(void);
/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
```

```
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_TIM2_Init();
  MX_TIM3_Init();
  /* USER CODE BEGIN 2 */
  // TODO: Start TIM3 in PWM mode on channel 3;
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
  TIM3->CCR3 = (1<<15);
  // TODO: Start TIM2 in Output Compare (OC) mode on channel 1
  HAL_TIM_OC_Start(&htim2, TIM_CHANNEL_1);
  // TODO: Start DMA in IT mode on TIM2->CH1. Source is LUT and Dest is TIM3->CCR3; start with Sine LUT
  HAL_DMA_Start_IT(&hdma_tim2_ch1, Sin_LUT, DestAddress, NS);
  // TODO: Write current waveform to LCD(Sine is the first waveform)
  init_LCD();
  lcd_command(CLEAR);
  lcd_update_line("Sine");
  // TODO: Enable DMA (start transfer from LUT to CCR)
  __HAL_TIM_ENABLE_DMA(&htim2, TIM_DMA_CC1);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
```

```
  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM2_Init(void)
{

  /* USER CODE BEGIN TIM2_Init 0 */

  /* USER CODE END TIM2_Init 0 */

  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM2_Init 1 */

  /* USER CODE END TIM2_Init 1 */
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 0;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
```

```
//  htim2.Init.Period = 4294967295;
  htim2.Init.Period = TIM2_Ticks;//#
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_OC_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_TIMING;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_OC_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */
  /* TIM2_CH1 DMA Init */
  __HAL_RCC_DMA1_CLK_ENABLE();

  hdma_tim2_ch1.Instance = DMA1_Stream5;
  hdma_tim2_ch1.Init.Channel = DMA_CHANNEL_3;         // TIM2_CH1 is on channel 3
  hdma_tim2_ch1.Init.Direction = DMA_MEMORY_TO_PERIPH; // Memory -> TIM3->CCR3
  hdma_tim2_ch1.Init.PeriphInc = DMA_PINC_DISABLE;    // Peripheral address fixed
  hdma_tim2_ch1.Init.MemInc = DMA_MINC_ENABLE;        // Memory address increments
  hdma_tim2_ch1.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
  hdma_tim2_ch1.Init.MemDataAlignment = DMA_MDATAALIGN_WORD;
  hdma_tim2_ch1.Init.Mode = DMA_CIRCULAR;            // Repeat LUT automatically
  hdma_tim2_ch1.Init.Priority = DMA_PRIORITY_HIGH;
  hdma_tim2_ch1.Init.FIFOMode = DMA_FIFOMODE_DISABLE;

  if (HAL_DMA_Init(&hdma_tim2_ch1) != HAL_OK)
  {
      Error_Handler();
  }

  /* Link DMA handle to TIM2 handle */
  __HAL_LINKDMA(&htim2, hdma[TIM_DMA_ID_CC1], hdma_tim2_ch1);
  /* USER CODE END TIM2_Init 2 */
```

```
}

/**
  * @brief TIM3 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM3_Init(void)
{

  /* USER CODE BEGIN TIM3_Init 0 */

  /* USER CODE END TIM3_Init 0 */

  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM3_Init 1 */

  /* USER CODE END TIM3_Init 1 */
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 0;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
//  htim3.Init.Period = 65535;
  htim3.Init.Period = (1<<12)-1;//#
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
//  sConfigOC.Pulse = 1;//#
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
```

```
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */

  /* USER CODE END TIM3_Init 2 */
  HAL_TIM_MspPostInit(&htim3);

}

/**
  * Enable DMA controller clock
  */
static void MX_DMA_Init(void)
{

  /* DMA controller clock enable */
  __HAL_RCC_DMA1_CLK_ENABLE();

  /* DMA interrupt init */
  /* DMA1_Stream5_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn);

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  // -------------------------------
  // LCD pins configuration
  // -------------------------------
  // Configure PC14 (RS) and PC15 (E) as output push-pull
  GPIO_InitStruct.Pin = GPIO_PIN_14 | GPIO_PIN_15;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  // Configure PB8 (D4) and PB9 (D5) as output push-pull
```

```
  GPIO_InitStruct.Pin = GPIO_PIN_8 | GPIO_PIN_9;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  // Configure PA12 (D6) and PA15 (D7) as output push-pull
  GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_15;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  // Set all LCD pins LOW initially
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14 | GPIO_PIN_15, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12 | GPIO_PIN_15, GPIO_PIN_RESET);


  // -------------------------------
  // Button0 configuration (PA0)
  // -------------------------------
  GPIO_InitStruct.Pin = Button0_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING; // Interrupt on rising edge
  GPIO_InitStruct.Pull = GPIO_PULLUP;         // Use pull-up resistor
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  // Enable and set EXTI line 0 interrupt priority
  HAL_NVIC_SetPriority(EXTI0_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(EXTI0_IRQn);

  /* USER CODE END MX_GPIO_Init_2 */

}

/* USER CODE BEGIN 4 */
void EXTI0_IRQHandler(void){

  // TODO: Debounce using HAL_GetTick()
  if (HAL_GetTick()-prev_tick>500) { //if more than 0.5 seconds have passed
            prev_tick = HAL_GetTick();

  // TODO: Disable DMA transfer and abort IT, then start DMA in IT mode with new LUT and re-enable transfer
  // HINT: Consider using C's "switch" function to handle LUT changes
  __HAL_TIM_DISABLE_DMA(&htim2,TIM_DMA_CC1);
  HAL_DMA_Abort_IT(&hdma_tim2_ch1);
  waveform = (waveform+1)%6;
  switch (waveform) {
  case 0:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Sin_LUT, DestAddress, NS);
            lcd_update_line("Sine");
            break;
  case 1:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Saw_LUT, DestAddress, NS);
            lcd_update_line("Sawtooth");
            break;
  case 2:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Triangle_LUT, DestAddress, NS);
            lcd_update_line("Triangular");
            break;
```

```
  case 3:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Piano_LUT, DestAddress, NS);
            lcd_update_line("Piano");
            break;
  case 4:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Guitar_LUT, DestAddress, NS);
            lcd_update_line("Guitar");
            break;
  case 5:
            HAL_DMA_Start_IT(&hdma_tim2_ch1, Drum_LUT, DestAddress, NS);
            lcd_update_line("Drum");
            break;
  }
  __HAL_TIM_ENABLE_DMA(&htim2, TIM_DMA_CC1);
}


HAL_GPIO_EXTI_IRQHandler(Button0_Pin); // Clear interrupt flags
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```